

Mahatma Education Society's
PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE
(Autonomous)
Re-accredited "A" Grade by NAAC (3rd Cycle)



Project Completion Certificate

THIS IS TO CERTIFY THAT
Krishna Kamlesh Singh
of FY BSC has completed the project titled
**PyUmbra: A Python-Based Umbrella Framework for Weather
and Environmental Insights**
under our guidance and supervision during the
academic year 2025-26 in the department of Computer
Science.

Project Guide

Course
Coordinator

Head of the
Department



PyUmbra

INTRODUCTION

PyUmbra: A Python-Based Umbrella Framework for Weather and Environmental Insights



Abstract:

PyUmbra is a Python-based project designed as an umbrella framework that combines multiple weather and environment-related features into a single reliable solution. Just as an umbrella protects us from uncertain weather, PyUmbra provides users with comprehensive insights into climatic and environmental conditions.

The project has been developed in **five modules**, where each module incrementally adds new features. This approach demonstrates step-by-step programming development, modularity, and scalability.

Module-Wise Functionality

Module 1:

- Input: City name
- Output:
 - City Name with Country
 - Temperature
 - Feels Like Temperature
 - Humidity

Module 2:

- Input: City name
- Output:
 - City Name with Country
 - Temperature
 - Feels Like Temperature
 - Humidity
 - **Air Quality Index (AQI)**

Module 3:

- Input: City name
- Output:
 - City Name with Country
 - Temperature
 - Feels Like Temperature
 - Humidity
 - **Particulate Matter (PM levels)**
 - **Wind Speed & Direction**

Module 4:

- Input: City name
- Output:
 - City Name with Country
 - Temperature
 - Feels Like Temperature
 - Humidity
 - **Particulate Matter (PM levels)**
 - **Wind Speed & Direction**
 - **Detected Emergencies (Environment/Weather Alerts)**

Module 5 (Final Module):

- Input: City name
- Output:
 - City Name with Country
 - Temperature
 - Feels Like Temperature
 - Humidity
 - **Particulate Matter (PM levels)**
 - **Wind Speed & Direction**
 - **Detected Emergencies**
 - **Advisories / Safety Guidelines**
 - **Emergency & Government Helpline Numbers (Country-Level)**

SWOT Analysis of PyUmbra

Strengths (S):

- **Modular Design:** Built in five progressive modules, showing stepwise learning and scalability.
- **Comprehensive Coverage:** Provides multiple insights (temperature, humidity, AQI, PM levels, wind, emergencies, advisories).
- **User-Centric:** Simple input (city name) and clear outputs, making it easy to use.
- **Practical Relevance:** Weather, air quality, and emergency alerts are highly relevant in today's world.
- **Python-Based:** Utilizes Python's strong libraries and APIs for real-time environmental data.

Weaknesses (W):

- **API/Internet Dependency:** Requires internet and APIs for real-time data; won't work offline.
- **Data Accuracy Limitation:** Accuracy depends on external data sources (APIs), not the program itself.
- **Scalability Challenges:** More features may lead to complex code maintenance.
- **Basic UI:** If command-line based, it may lack a user-friendly graphical interface.

Opportunities (O):

- **Integration with IoT:** Can be expanded for smart home/weather devices.
- **Mobile & Web App Development:** Can be turned into an Android/iOS/web app for wider use.
- **Disaster Management Use:** Useful for government, NGOs, and emergency services.
- **Educational Tool:** Can be used to teach modular programming and applied Python in real-world scenarios.
- **Data Visualization:** Adding graphs/charts (e.g., AQI trends, wind speed charts) will make it more powerful.

Threats (T):

- **API Access Restrictions:** Free APIs may have limits or may stop working in future.
- **Competition:** Many weather apps already exist (AccuWeather, AQI apps, Google Weather).
- **Data Privacy Issues:** If extended to collect user location, privacy concerns may arise.
- **System Compatibility:** Might face issues when integrating with different platforms or versions of Python.

Conclusion:

PyUmbra is a modular, Python-based project that delivers practical weather and air quality insights with simple usability. While API dependency and basic UI limit its scalability, it remains a strong educational and real-world tool. With opportunities in IoT, apps, and visualization, PyUmbra has the potential to grow into a robust environmental monitoring solution.

Project Details:

- **Course:** B.Sc. Computer Science (FY)
- **Subject:** Python Programming (CA-2)
- **Academic Year:** 2025–26
- **Submitted by:** *Krishna Kamlesh Singh (Roll No. 1400)*
- **Division:** C (Computer Science Section)

MODULE 1

SOURCE CODE:

```
import requests

from tabulate import tabulate

# ---- API KEY (OpenWeather) ----

OPENWEATHER_API = "2ea555c0e8800f6d30d000bed5888a52"

# ---- Endpoints ----

WEATHER_URL = "http://api.openweathermap.org/data/2.5/weather"

# ---- Data fetcher ----

def get_weather(city: str):

    try:

        r = requests.get(WEATHER_URL, params={

            "q": city, "appid": OPENWEATHER_API, "units": "metric"

        }, timeout=12)

        r.raise_for_status()

        return r.json()

    except Exception:

        return None

# ---- Main program ----

def main():

    city = input("Enter city name: ").strip()

    if not city:

        print("City is required.")

        return

    weather = get_weather(city)

    if not weather or weather.get("cod") != 200:

        print("City not found or OpenWeather issue.")

        return

    # Basic info
```

```

city_name = weather.get("name", city)

country_code = weather.get("sys", {}).get("country", "")

temp = weather.get("main", {}).get("temp")

feels = weather.get("main", {}).get("feels_like")

humidity = weather.get("main", {}).get("humidity", None)

# ---- Output in table ----

table = [

    ["City", f"{city_name}, {country_code}"],

    ["Temperature (°C)", f"{temp:.1f}"],

    ["Feels Like (°C)", f"{feels:.1f}"],

    ["Humidity (%)", f"{humidity}"],

]

print("\n--- CITY REPORT ---")

print(tabulate(table, headers=["Parameter", "Value"], tablefmt="fancy_grid"))

if __name__ == "__main__":

    main()

```

OUTPUT:

```

PS C:\Users\admin\Documents\PCACS PYTHON\PROJECT SEM 1>
Documents\PCACS PYTHON\PROJECT SEM 1\MODULE 1.py
Enter city name: Mumbai

--- CITY REPORT ---

```

Parameter	Value
City	Mumbai, IN
Temperature (°C)	26.1
Feels Like (°C)	26.1
Humidity (%)	91

Enter city name: Dubai

--- CITY REPORT ---

Parameter	Value
City	Dubai, AE
Temperature (°C)	38.3
Feels Like (°C)	45.3
Humidity (%)	58

Enter city name: Varanasi

--- CITY REPORT ---

Parameter	Value
City	Varanasi, IN
Temperature (°C)	31.9
Feels Like (°C)	39.0
Humidity (%)	67

Enter city name: Tokyo

--- CITY REPORT ---

Parameter	Value
City	Tokyo, JP
Temperature (°C)	31.2
Feels Like (°C)	35.1
Humidity (%)	59

MODULE 2

SOURCE CODE:

```
import requests

from tabulate import tabulate

# ---- API KEY (OpenWeather) ----

OPENWEATHER_API = "2ea555c0e8800f6d30d000bed5888a52"

# ---- Helpline dictionary (country-level) ----

helplines = {

    "IN": "India Helpline: 112 / Police: 100 / Ambulance: 108 / Fire: 101",

    "GB": "UK Emergency: 999",

    "US": "USA Emergency: 911",

    "FR": "France Emergency: 112",

    "DE": "Germany Emergency: 112",

    "default": "Local Emergency Number: 112"

}

# ---- Endpoints ----

WEATHER_URL = "http://api.openweathermap.org/data/2.5/weather"

AIR_URL = "http://api.openweathermap.org/data/2.5/air_pollution"

# ---- AQI helpers (US EPA mapping) ----

def _linear_map(C, Clow, Chigh, Ilow, Ihigh):

    return (Ihigh - Ilow) * (C - Clow) / (Chigh - Clow) + Ilow

def aqi_pm25(pm25: float) -> int:

    bps = [

        (0.0, 12.0, 0, 50),

        (12.1, 35.4, 51, 100),

        (35.5, 55.4, 101, 150),

        (55.5, 150.4, 151, 200),

        (150.5, 250.4, 201, 300),

        (250.5, 350.4, 301, 400),

        (350.5, 500.4, 401, 500),
```



```

]

for Clow, Chigh, Ilow, Ihigh in bps:

    if Clow <= pm25 <= Chigh:

        return int(round(_linear_map(pm25, Clow, Chigh, Ilow, Ihigh)))

return 500

```

```

def aqi_pm10(pm10: float) -> int:

```

```

    bps = [

        (0, 54, 0, 50),

        (55, 154, 51, 100),

        (155, 254, 101, 150),

        (255, 354, 151, 200),

        (355, 424, 201, 300),

        (425, 504, 301, 400),

        (505, 604, 401, 500),

```

```

    ]

    for Clow, Chigh, Ilow, Ihigh in bps:

        if Clow <= pm10 <= Chigh:

            return int(round(_linear_map(pm10, Clow, Chigh, Ilow, Ihigh)))

    return 500

```

```

def aqi_category(index: int) -> str:

```

```

    if index <= 50: return "Good"

    if index <= 100: return "Moderate"

    if index <= 150: return "Unhealthy for Sensitive Groups"

    if index <= 200: return "Unhealthy"

    if index <= 300: return "Very Unhealthy"

    return "Hazardous"

```

```

# ---- Data fetchers ----

```

```

def get_weather(city: str):

```

```

    try:

        r = requests.get(WEATHER_URL, params={

            "q": city, "appid": OPENWEATHER_API, "units": "metric"

```

```

    }, timeout=12)

    r.raise_for_status()

    return r.json()

except Exception:

    return None


def get_aqi(lat: float, lon: float):

    try:

        r = requests.get(AIR_URL, params={"lat": lat, "lon": lon, "appid": OPENWEATHER_API}, timeout=12)

        r.raise_for_status()

        data = r.json()

        comp = data["list"][0]["components"]

        pm25 = comp.get("pm2_5", 0.0)

        pm10 = comp.get("pm10", 0.0)

        index = max(aqi_pm25(pm25), aqi_pm10(pm10))

        return index, comp

    except Exception:

        return None, None


# ---- Main program ----

def main():

    city = input("Enter city name: ").strip()

    if not city:

        print("City is required.")

        return

    weather = get_weather(city)

    if not weather or weather.get("cod") != 200:

        print("City not found or OpenWeather issue.")

        return

    # Extract basics

    city_name = weather.get("name", city)

    country_code = weather.get("sys", {}).get("country", "")

    temp = weather.get("main", {}).get("temp")

```

```

feels = weather.get("main", {}).get("feels_like")

humidity = weather.get("main", {}).get("humidity", None)

lat = weather.get("coord", {}).get("lat")

lon = weather.get("coord", {}).get("lon")


# AQI

aqi_index = None

if lat is not None and lon is not None:

    aqi_index, _ = get_aqi(lat, lon)


# ---- Output in table ----

table = [

    ["City", f"{city_name}, {country_code}"],

    ["Temperature (°C)", f"{temp:.1f}"],

    ["Feels Like (°C)", f"{feels:.1f}"],

    ["Humidity (%)", f"{humidity}"],

]

if aqi_index is not None:

    table.append(["Air Quality Index", f"{aqi_index} — {aqi_category(aqi_index)}"])

else:

    table.append(["Air Quality Index", "Not available"])


# Emergency Helpline

helpline = helplines.get(country_code, helplines["default"])

table.append(["Emergency Helpline", helpline])


print("\n--- CITY REPORT ---")

print(tabulate(table, headers=["Parameter", "Value"], tablefmt="fancy_grid"))


if __name__ == "__main__":

    main()

```

OUTPUT:

```
PS C:\Users\admin\Documents\PCACS PYTHON\PROJECT SEM 1>
Documents\PCACS PYTHON\PROJECT SEM 1\MODULE 2.py"
Enter city name: Sydney
```

--- CITY REPORT ---

Parameter	Value
City	Sydney, AU
Temperature (°C)	10.4
Feels Like (°C)	9.9
Humidity (%)	92
Air Quality Index	2 – Good
Emergency Helpline	Local Emergency Number: 112

```
Enter city name: Kawasaki
```

--- CITY REPORT ---

Parameter	Value
City	Kawasaki, JP
Temperature (°C)	28.6
Feels Like (°C)	33.5
Humidity (%)	79
Air Quality Index	56 – Moderate
Emergency Helpline	Local Emergency Number: 112

```
Enter city name: New Delhi
```

--- CITY REPORT ---

Parameter	Value
City	New Delhi, IN
Temperature (°C)	30.6
Feels Like (°C)	34.0
Humidity (%)	60
Air Quality Index	54 – Moderate
Emergency Helpline	India Helpline: 112 / Police: 100 / Ambulance: 108 / Fire: 101

MODULE 3

SOURCE CODE:

```
import requests

from tabulate import tabulate

# ---- API KEY (OpenWeather) ----

OPENWEATHER_API = "2ea555c0e8800f6d30d000bed5888a52"

# ---- Endpoints ----

WEATHER_URL = "http://api.openweathermap.org/data/2.5/weather"

AIR_URL = "http://api.openweathermap.org/data/2.5/air_pollution"

# ---- AQI helpers (US EPA mapping) ----

def _linear_map(C, Clow, Chigh, Ilow, Ihigh):

    return (Ihigh - Ilow) * (C - Clow) / (Chigh - Clow) + Ilow

def aqi_pm25(pm25: float) -> int:

    bps = [

        (0.0, 12.0, 0, 50),

        (12.1, 35.4, 51, 100),

        (35.5, 55.4, 101, 150),

        (55.5, 150.4, 151, 200),

        (150.5, 250.4, 201, 300),

        (250.5, 350.4, 301, 400),

        (350.5, 500.4, 401, 500),

    ]

    for Clow, Chigh, Ilow, Ihigh in bps:

        if Clow <= pm25 <= Chigh:

            return int(round(_linear_map(pm25, Clow, Chigh, Ilow, Ihigh)))

    return 500

def aqi_pm10(pm10: float) -> int:

    bps = [

        (0, 54, 0, 50),

        (55, 154, 51, 100),
```

```
(155, 254, 101, 150),  
(255, 354, 151, 200),  
(355, 424, 201, 300),  
(425, 504, 301, 400),  
(505, 604, 401, 500),
```

```
]
```

```
for Clow, Chigh, Ilow, Ihigh in bps:
```

```
    if Clow <= pm10 <= Chigh:
```

```
        return int(round(_linear_map(pm10, Clow, Chigh, Ilow, Ihigh)))
```

```
    return 500
```

```
def aqi_category(index: int) -> str:
```

```
    if index <= 50: return "Good"
```

```
    if index <= 100: return "Moderate"
```

```
    if index <= 150: return "Unhealthy for Sensitive Groups"
```

```
    if index <= 200: return "Unhealthy"
```

```
    if index <= 300: return "Very Unhealthy"
```

```
    return "Hazardous"
```

```
# ---- Data fetchers ----
```

```
def get_weather(city: str):
```

```
    try:
```

```
        r = requests.get(WEATHER_URL, params={
```

```
            "q": city, "appid": OPENWEATHER_API, "units": "metric"
```

```
        }, timeout=12)
```

```
        r.raise_for_status()
```

```
        return r.json()
```

```
    except Exception:
```

```
        return None
```

```
def get_aqi(lat: float, lon: float):
```

```
    try:
```

```
        r = requests.get(AIR_URL, params={"lat": lat, "lon": lon, "appid": OPENWEATHER_API}, timeout=12)
```

```
        r.raise_for_status()
```

```

    data = r.json()

    comp = data["list"][0]["components"]

    pm25 = comp.get("pm2_5", 0.0)

    pm10 = comp.get("pm10", 0.0)

    index = max(aqi_pm25(pm25), aqi_pm10(pm10))

    return index, comp

except Exception:

    return None, None

# ---- Main program ----

def main():

    city = input("Enter city name: ").strip()

    if not city:

        print("City is required.")

        return

    weather = get_weather(city)

    if not weather or weather.get("cod") != 200:

        print("City not found or OpenWeather issue.")

        return

    # Basic info

    city_name = weather.get("name", city)

    temp = weather.get("main", {}).get("temp")

    feels = weather.get("main", {}).get("feels_like")

    wind = weather.get("wind", {}).get("speed", 0)

    humidity = weather.get("main", {}).get("humidity", None)

    lat = weather.get("coord", {}).get("lat")

    lon = weather.get("coord", {}).get("lon")

    # AQI

    aqi_index, components = (None, None)

    if lat is not None and lon is not None:

        aqi_index, components = get_aqi(lat, lon)

```

```

# ---- Output in Table ----

table = [
    ["City", city_name],
    ["Temperature (°C)", f"{temp:.1f}"],
    ["Feels Like (°C)", f"{feels:.1f}"],
    ["Humidity (%)", f"{humidity}"],
    ["Wind Speed (m/s)", f"{wind}"],
]

if aqi_index is not None:
    table.append(["Air Quality Index", f"{aqi_index} — {aqi_category(aqi_index)}"])
    if components:
        pm25 = components.get("pm2_5", "N/A")
        pm10 = components.get("pm10", "N/A")
        table.append(["PM2.5 (µg/m³)", pm25])
        table.append(["PM10 (µg/m³)", pm10])
    else:
        table.append(["Air Quality Index", "Not available"])

print("\n--- CITY WEATHER HIGHLIGHTS ---")
print(tabulate(table, headers=["Parameter", "Value"], tablefmt="fancy_grid"))

if __name__ == "__main__":
    main()

```

OUTPUT:


```
PS C:\Users\admin\Documents\PCACS PYTHON\PROJECT SEM 1>
Documents\PCACS PYTHON\PROJECT SEM 1\MODULE 3.py"
Enter city name: New York
```

--- CITY WEATHER HIGHLIGHTS ---

Parameter	Value
City	New York
Temperature (°C)	19.2
Feels Like (°C)	19.1
Humidity (%)	74
Wind Speed (m/s)	7.28
Air Quality Index	25 – Good
PM2.5 (µg/m³)	5.95
PM10 (µg/m³)	6.63

```
Enter city name: Panaji
```

--- CITY WEATHER HIGHLIGHTS ---

Parameter	Value
City	Panjim
Temperature (°C)	27.6
Feels Like (°C)	31.2
Humidity (%)	81
Wind Speed (m/s)	6.02
Air Quality Index	17 – Good
PM2.5 (µg/m³)	4.12
PM10 (µg/m³)	8.96

MODULE 4

SOURCE CODE:

```
import requests

from tabulate import tabulate # pip install tabulate

# ---- API KEY (OpenWeather) ----

OPENWEATHER_API = "2ea555c0e8800f6d30d000bed5888a52"

# ---- Endpoints ----

WEATHER_URL = "http://api.openweathermap.org/data/2.5/weather"

AIR_URL = "http://api.openweathermap.org/data/2.5/air_pollution"

# ---- AQI helpers (US EPA mapping) ----

def _linear_map(C, Clow, Chigh, Ilow, Ihigh):

    return (Ihigh - Ilow) * (C - Clow) / (Chigh - Clow) + Ilow

def aqi_pm25(pm25: float) -> int:

    bps = [

        (0.0, 12.0, 0, 50),

        (12.1, 35.4, 51, 100),

        (35.5, 55.4, 101, 150),

        (55.5, 150.4, 151, 200),

        (150.5, 250.4, 201, 300),

        (250.5, 350.4, 301, 400),

        (350.5, 500.4, 401, 500),

    ]

    for Clow, Chigh, Ilow, Ihigh in bps:

        if Clow <= pm25 <= Chigh:

            return int(round(_linear_map(pm25, Clow, Chigh, Ilow, Ihigh)))

    return 500

def aqi_pm10(pm10: float) -> int:

    bps = [

        (0, 54, 0, 50),

        (55, 154, 51, 100),
```

```
(155, 254, 101, 150),  
(255, 354, 151, 200),  
(355, 424, 201, 300),  
(425, 504, 301, 400),  
(505, 604, 401, 500),
```

```
]
```

```
for Clow, Chigh, Ilow, Ihigh in bps:
```

```
    if Clow <= pm10 <= Chigh:
```

```
        return int(round(_linear_map(pm10, Clow, Chigh, Ilow, Ihigh)))
```

```
    return 500
```

```
def aqi_category(index: int) -> str:
```

```
    if index <= 50: return "Good"
```

```
    if index <= 100: return "Moderate"
```

```
    if index <= 150: return "Unhealthy (Sensitive)"
```

```
    if index <= 200: return "Unhealthy"
```

```
    if index <= 300: return "Very Unhealthy"
```

```
    return "Hazardous"
```

```
# ---- Data fetchers ----
```

```
def get_weather(city: str):
```

```
    try:
```

```
        r = requests.get(WEATHER_URL, params={
```

```
            "q": city, "appid": OPENWEATHER_API, "units": "metric"
```

```
        }, timeout=12)
```

```
        r.raise_for_status()
```

```
        return r.json()
```

```
    except Exception:
```

```
        return None
```

```
def get_aqi(lat: float, lon: float):
```

```
    try:
```

```
        r = requests.get(AIR_URL, params={"lat": lat, "lon": lon, "appid": OPENWEATHER_API}, timeout=12)
```

```
        r.raise_for_status()
```

```

data = r.json()

comp = data["list"][0]["components"]

pm25 = comp.get("pm2_5", 0.0)

pm10 = comp.get("pm10", 0.0)

index = max(aqi_pm25(pm25), aqi_pm10(pm10))

return index, comp

except Exception:

    return None, None

# ---- Emergency detection ----

def detect_emergency(weather_json, aqi_index):

    detected = []

    if not weather_json:

        return detected

    main_cond = weather_json["weather"][0].get("main", "").lower()

    desc = weather_json["weather"][0].get("description", "").lower()

    temp = weather_json["main"].get("temp", None)

    wind = weather_json.get("wind", {}).get("speed", 0)

    if any(k in main_cond for k in ["thunderstorm", "storm"]) or "heavy" in desc:

        detected.append("Severe Storm / Heavy Rain")

    if "rain" in main_cond and wind >= 12:

        detected.append("Rain with Strong Wind")

    if temp is not None and temp >= 40:

        detected.append("Heatwave")

    if temp is not None and temp <= 5:

        detected.append("Cold Wave")

    if wind >= 15:

        detected.append("High Wind")

    if aqi_index is not None and aqi_index >= 151:

        detected.append("Severe Air Pollution")

    return list(dict.fromkeys(detected))

```

```
# ---- Main program ----
```

```
def main():
```

```
    city = input("Enter city name: ").strip()
```

```
    if not city:
```

```
        print("City is required.")
```

```
        return
```

```
    weather = get_weather(city)
```

```
    if not weather or weather.get("cod") != 200:
```

```
        print("City not found or OpenWeather issue.")
```

```
        return
```

```
    city_name = weather.get("name", city)
```

```
    country_code = weather.get("sys", {}).get("country", "")
```

```
    temp = weather.get("main", {}).get("temp")
```

```
    feels = weather.get("main", {}).get("feels_like")
```

```
    humidity = weather.get("main", {}).get("humidity", None)
```

```
    wind = weather.get("wind", {}).get("speed", 0)
```

```
    lat = weather.get("coord", {}).get("lat")
```

```
    lon = weather.get("coord", {}).get("lon")
```

```
    aqi_index, components = (None, None)
```

```
    if lat is not None and lon is not None:
```

```
        aqi_index, components = get_aqi(lat, lon)
```

```
    emergencies = detect_emergency(weather, aqi_index)
```

```
# ---- OUTPUT in Table ----
```

```
table_data = [
```

```
    ["City", f"{city_name}, {country_code}"],
```

```
    ["Temperature", f"{temp:.1f}°C (Feels like {feels:.1f}°C)"],
```

```
    ["Humidity", f"{humidity}%"],
```

```
    ["Wind Speed", f"{wind} m/s"],
```

```
]
```

```

if aqi_index is not None:

    table_data.append(["AQI", f"{aqi_index} ({aqi_category(aqi_index)})"])

    if components:

        table_data.append(["PM2.5 / PM10", f"{components.get('pm2_5', 'N/A')} µg/m³ / {components.get('pm10',
'N/A')} µg/m³"])

    else:

        table_data.append(["AQI", "Not available"])

if emergencies:

    table_data.append(["Emergencies", ", ".join(emergencies)])

else:

    table_data.append(["Emergencies", "No immediate emergencies detected"])

print("\n===== CITY REPORT =====")

print(tabulate(table_data, headers=["Parameter", "Value"], tablefmt="fancy_grid"))

if __name__ == "__main__":

    main()

```

OUTPUT:

```

PS C:\Users\admin\Documents\PCACS PYTHON\PROJECT SEM 1>
Documents/PCACS PYTHON/PROJECT SEM 1/MODULE 4.py
Enter city name: Srinagar

===== CITY REPORT =====

```

Parameter	Value
City	Srinagar, IN
Temperature	18.9°C (Feels like 19.1°C)
Humidity	87%
Wind Speed	1.57 m/s
AQI	33 (Good)
PM2.5 / PM10	7.89 µg/m³ / 8.53 µg/m³
Emergencies	No immediate emergencies detected

Enter city name: Patos

===== CITY REPORT =====

Parameter	Value
City	Patos, BR
Temperature	28.0°C (Feels like 27.3°C)
Humidity	33%
Wind Speed	4.28 m/s
AQI	5 (Good)
PM2.5 / PM10	1.15 µg/m³ / 2.5 µg/m³
Emergencies	No immediate emergencies detected

Source: [OpenWeatherMap](#) and [AirVisual](#)

Report generated on 2024-09-27 15:00 UTC

Report ID: PATOS-20240927-1500

Report Version: 1.0

Report Status: Success

Report Author: WeatherData

Report License: CC BY-NC-SA

Report Contact: [support@weatherdata.com](#)

Report Footer: All rights reserved.

Report Disclaimer: This report is for informational purposes only.

Report Privacy Policy: [Privacy Policy](#)

Report Terms of Service: [Terms of Service](#)

Report Copyright: © 2024 WeatherData

Report All Rights Reserved

Report No Warranties

Report No Liability

Report No Damages

Report No Consequences

Report No Results

Report No Guarantees

Report No Promises

Report No Agreements

Report No Contracts

Report No Obligations

Report No Responsibilities

Report No Duties

Report No Powers

Report No Privileges

Report No Immunities

Report No Exemptions

Report No Exceptions

Report No Defenses

Report No Claims

Report No Causes of Action

Report No Remedies

Report No Relief

Report No Damages

Report No Compensation

Report No Settlements

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

Report No Litigation

Report No Proceedings

Report No Hearings

Report No Trials

Report No Judgments

Report No Verdicts

Report No Findings

Report No Conclusions

Report No Recommendations

Report No Suggestions

Report No Advice

Report No Opinions

Report No Comments

Report No Statements

Report No Declarations

Report No Assertions

Report No Allegations

Report No Accusations

Report No Charges

Report No Suits

</

MODULE 5

SOURCE CODE:

```
import requests

# ---- API KEY ----

OPENWEATHER_API = "2ea555c0e8800f6d30d000bed5888a52"

# ---- Helplines ----

helplines = {

    "IN": "India Helpline: 112 / Police: 100 / Ambulance: 108 / Fire: 101",

    "GB": "UK Emergency: 999",

    "US": "USA Emergency: 911",

    "FR": "France Emergency: 112",

    "DE": "Germany Emergency: 112",

    "default": "Local Emergency Number: 112"

}

# ---- Endpoints ----

WEATHER_URL = "http://api.openweathermap.org/data/2.5/weather"

AIR_URL = "http://api.openweathermap.org/data/2.5/air_pollution"

REST_ALPHA = "https://restcountries.com/v3.1/alpha/"

# ---- AQI helpers ----

def _linear_map(C, Clow, Chigh, Ilow, Ihigh):

    return (Ihigh - Ilow) * (C - Clow) / (Chigh - Clow) + Ilow

def aqi_pm25(pm25: float) -> int:

    bps = [

        (0.0, 12.0, 0, 50),

        (12.1, 35.4, 51, 100),

        (35.5, 55.4, 101, 150),

        (55.5, 150.4, 151, 200),

        (150.5, 250.4, 201, 300),

        (250.5, 350.4, 301, 400),

        (350.5, 500.4, 401, 500),
```



```

]

for Clow, Chigh, Ilow, Ihigh in bps:

    if Clow <= pm25 <= Chigh:

        return int(round(_linear_map(pm25, Clow, Chigh, Ilow, Ihigh)))

return 500

```

```

def aqi_pm10(pm10: float) -> int:

```

```

    bps = [

        (0, 54, 0, 50),

        (55, 154, 51, 100),

        (155, 254, 101, 150),

        (255, 354, 151, 200),

        (355, 424, 201, 300),

        (425, 504, 301, 400),

        (505, 604, 401, 500),

```

```

    ]

```

```

    for Clow, Chigh, Ilow, Ihigh in bps:

        if Clow <= pm10 <= Chigh:

            return int(round(_linear_map(pm10, Clow, Chigh, Ilow, Ihigh)))

    return 500

```

```

def aqi_category(index: int) -> str:

```

```

    if index <= 50: return "Good"

    if index <= 100: return "Moderate"

    if index <= 150: return "Unhealthy (Sensitive)"

    if index <= 200: return "Unhealthy"

    if index <= 300: return "Very Unhealthy"

    return "Hazardous"

```

```

# ---- Fetchers ----

```

```

def get_weather(city: str):

```

```

    try:

        r = requests.get(WEATHER_URL, params={

            "q": city, "appid": OPENWEATHER_API, "units": "metric"

```

```

    }, timeout=12)

    r.raise_for_status()

    return r.json()

except Exception:

    return None


def get_aqi(lat: float, lon: float):

    try:

        r = requests.get(AIR_URL, params={"lat": lat, "lon": lon, "appid": OPENWEATHER_API}, timeout=12)

        r.raise_for_status()

        data = r.json()

        comp = data["list"][0]["components"]

        pm25 = comp.get("pm2_5", 0.0)

        pm10 = comp.get("pm10", 0.0)

        index = max(aqi_pm25(pm25), aqi_pm10(pm10))

        return index, comp

    except Exception:

        return None, None


def get_population(country_code: str):

    try:

        r = requests.get(REST_ALPHA + country_code, timeout=12)

        r.raise_for_status()

        j = r.json()

        return int(j[0].get("population", 0))

    except Exception:

        return None


# ---- Emergency + Advisories ----

def detect_emergency(weather_json, aqi_index):

    detected = []

    main_cond = weather_json["weather"][0].get("main", "").lower()

    desc = weather_json["weather"][0].get("description", "").lower()

    temp = weather_json["main"].get("temp", None)

```

```

wind = weather_json.get("wind", {}).get("speed", 0)

if any(k in main_cond for k in ["thunderstorm", "storm"]) or "heavy" in desc:
    detected.append("Severe Storm / Heavy Rain")

if "rain" in main_cond and wind >= 12:
    detected.append("Rain with Strong Wind")

if temp is not None and temp >= 40:
    detected.append("Heatwave")

if temp is not None and temp <= 5:
    detected.append("Cold Wave")

if wind >= 15:
    detected.append("High Wind")

if aqi_index is not None and aqi_index >= 151:
    detected.append("Severe Air Pollution")

return list(dict.fromkeys(detected)) # unique order

def build_advisories(weather_json, aqi_index):
    adv = []

    temp = weather_json["main"].get("temp", None)
    desc = weather_json["weather"][0].get("description", "").lower()
    wind = weather_json.get("wind", {}).get("speed", 0)
    hum = weather_json["main"].get("humidity", 0)

    if "rain" in desc or "storm" in desc:
        adv += ["Carry umbrella/raincoat", "Avoid flooded roads"]

    if temp is not None and temp >= 35:
        adv += ["Stay hydrated", "Avoid outdoor activity midday"]

    if temp is not None and temp <= 10:
        adv += ["Dress warmly", "Check on elderly/infants"]

    if wind >= 15:
        adv += ["Secure loose outdoor items"]

    if hum >= 85 and "rain" in desc:
        adv += ["Use waterproof footwear"]

    if aqi_index is not None:

```

```

if aqi_index >= 301:
    adv += ["Air hazardous — stay indoors"]
elif aqi_index >= 201:
    adv += ["Very unhealthy — use N95 mask"]
elif aqi_index >= 151:
    adv += ["Unhealthy — sensitive groups stay inside"]
elif aqi_index >= 101:
    adv += ["Moderate — avoid long outdoor exertion"]
else:
    adv += ["Air quality acceptable"]

```

```

final = []

```

```

for a in adv:

```

```

    if a not in final:
        final.append(a)
    if len(final) >= 5:
        break

```

```

while len(final) < 3:

```

```

    final.append("Keep emergency numbers handy.")

```

```

return final

```

```

# ---- Table Printers ----

```

```

def _print_table(rows, headers=("Parameter", "Value")):

```

```

    w1 = max(len(headers[0]), max((len(str(k)) for k, _ in rows), default=0))

```

```

    w2 = max(len(headers[1]), max((len(str(v)) for _, v in rows), default=0))

```

```

    line = "+" + "-"*(w1+2) + "+" + "-"*(w2+2) + "+"

```

```

    sep = "+" + "="*(w1+2) + "+" + "="*(w2+2) + "+"

```

```

    print(line)

```

```

    print(f"| {headers[0]:<{w1}} | {headers[1]:<{w2}} |")

```

```

    print(sep)

```

```

    for k, v in rows:

```

```

        print(f"| {k:<{w1}} | {v:<{w2}} |")

```

```

    print(line)

```

```

def _print_list_table(items, header):
    w = max(len(header), max((len(str(x)) for x in items), default=0))

    line = "+" + "-"*(w+2) + "+"
    sep = "+" + "="*(w+2) + "+"

    print(line)

    print(f"| {header:<{w}} |")

    print(sep)

    if items:
        for x in items:
            print(f"| {x:<{w}} |")
    else:
        print(f"| {'-':<{w}} |")

    print(line)

```

---- Main ----

```

def main():
    city = input("Enter city name: ").strip()
    weather = get_weather(city)

    if not weather or weather.get("cod") != 200:
        print("City not found.")
        return

    city_name = weather["name"]
    country = weather["sys"]["country"]
    temp = weather["main"]["temp"]
    feels = weather["main"]["feels_like"]
    humidity = weather["main"]["humidity"]
    desc = weather["weather"][0]["description"].title()
    wind = weather["wind"]["speed"]

    lat, lon = weather["coord"]["lat"], weather["coord"]["lon"]
    aqi_index, components = get_aqi(lat, lon)
    population = get_population(country)
    emergencies = detect_emergency(weather, aqi_index)

```

```

advisories = build_advisories(weather, aqi_index)

contact = helplines.get(country, helplines["default"])

# ---- Output ----

print("\n=== WEATHER & ENVIRONMENT REPORT ===\n")

_print_table([
    ("City", f"{city_name}, {country}"),
    ("Condition", desc),
    ("Temperature (°C)", temp),
    ("Feels Like (°C)", feels),
    ("Humidity (%)", humidity),
    ("Wind Speed (m/s)", wind),
    ("Population", population if population else "N/A")
])

if aqi_index is not None:
    _print_table([
        ("AQI Index", aqi_index),
        ("Category", aqi_category(aqi_index))
    ], headers=("Air Quality", "Value"))

if components:
    _print_table([(k.upper(), v) for k, v in components.items()],
        headers=("Pollutant", "µg/m³"))

_print_list_table(emergencies, "Detected Emergencies")
_print_list_table(advisories, "Safety Advisories")

_print_table([("Emergency Contacts", contact)], headers=("Country Helpline", "Number"))

if __name__ == "__main__":
    main()

```

OUTPUT:

Enter city name: Jaipur

Parameter	Value
City	Jaipur, IN
Condition	Broken Clouds
Temperature (°C)	31.15
Feels Like (°C)	34.42
Humidity (%)	57
Wind Speed (m/s)	4.82
Population	1380004385

Pollutant	µg/m³
CO	246.14
NO	0.18
NO2	5.96
O3	73.89
SO2	2.65
PM2_5	17.87
PM10	19.1
NH3	2.15

```
=====
| Safety Advisories |
=====
| Air quality acceptable |
| Keep emergency numbers handy. |
| Keep emergency numbers handy. |
```

Country Helpline	Number
Emergency Contacts	India Helpline: 112 / Police: 100 / Ambulance: 108 / Fire: 101

Enter city name: Tasmania

=== WEATHER & ENVIRONMENT REPORT ===

Parameter	Value
City	Tasmania, AU
Condition	Overcast Clouds
Temperature (°C)	3.6
Feels Like (°C)	-0.91
Humidity (%)	92
Wind Speed (m/s)	6.07
Population	25687041

Air Quality	Value
AQI Index	5
Category	Good

Pollutant	µg/m³
CO	67.85
NO	0
NO2	0.01
O3	60.58
SO2	0.04
PM2_5	1.31
PM10	1.96
NH3	0

Detected Emergencies
Cold Wave

Safety Advisories
Dress warmly
Check on elderly/infants
Air quality acceptable

Country Helpline	Number
Emergency Contacts	Local Emergency Number: 112

Conclusion

The project **PyUmbra: A Python-Based Umbrella Framework for Weather and Environmental Insights** demonstrates how modular programming, Python libraries, and API integrations can be combined to create a robust and user-friendly application.

To achieve this, I used essential Python libraries such as **requests** for interacting with live APIs and **tabulate** (or table-formatting methods) to present results in a clean, structured tabular form, ensuring readability and professional output for end users.

The framework was designed in **five incremental modules**, each building upon the previous one and adding **unique features** that expand the scope of information available to users:

- **Module 1** introduced the basic weather details — city, temperature, feels-like temperature, and humidity — giving users an immediate climate overview.
- **Module 2** enhanced the utility by incorporating **Air Quality Index (AQI)** data, linking health awareness with weather conditions.
- **Module 3** added **particulate matter (PM) levels** along with **wind speed and direction**, making the system more relevant for environmental monitoring and outdoor planning.
- **Module 4** focused on **detected emergencies**, identifying severe weather or environmental risks (storms, heavy rainfall, heatwaves, or high pollution levels) and warning users accordingly.
- **Module 5** completed the framework by providing **advisories, safety guidelines, and country-level emergency helpline numbers**, turning the system from an information provider into a **decision-support tool** for public safety.

By combining all these modules, PyUmbra evolved from a simple weather-checking tool into a **comprehensive environmental insight platform**. Each stage added meaningful and unique information that benefits users in different contexts — from everyday planning to emergency awareness.

Overall, the project highlights the importance of **modularity, scalability, and user-centric design** in Python development. It not only provides technical insights into data handling and API usage but also emphasizes practical real-world applications by delivering actionable information to users in a clear and structured manner.

Project Details:

- **Course:** B.Sc. Computer Science (FY)
- **Subject:** Python Programming (CA-2)
- **Academic Year:** 2025–26
- **Submitted by:** *Krishna Kamlesh Singh (Roll No. 1400)*
- **Division:** C (Computer Science Section)